

How to Build AI Agents

A practical guide for developers

From the simplest workflow that works to production-grade autonomy. What actually matters in 2026, and what to skip.

Last updated: May 2026

Contents

Most teams that set out to build an AI agent end up with one of two things: a chatbot with extra steps, or a fragile autonomous system that demos beautifully and falls apart in production. This guide is about avoiding both. It walks through what an agent actually is, the building blocks, how to choose between writing your own loop and adopting a framework, and the unglamorous reliability work that decides whether your agent ships or stays a prototype.

The good news for 2026: the models are good enough. The difference between an agent that works and one that doesn't is rarely the model you pick. It is the system you build around it.

1. Agent, chatbot, or workflow?

Before writing any code, get the category right, because it changes everything downstream.

- **Chatbot.** You ask, it answers. It waits for input and responds to a single turn. No initiative, no actions in the world.
- **Workflow.** A predefined sequence of steps, some of which call a model. The control flow is written by you, in code. The model fills in specific blanks, but it does not decide what happens next.
- **Agent.** The model decides what to do next at runtime. It plans, calls tools, observes results, and loops until the goal is met or it gives up. The control flow is dynamic and chosen by the model, not hardcoded by you.

This distinction, popularized by Anthropic's writing on effective agents, is the single most useful idea in the space. Both workflows and agents are legitimate. They differ in who owns the control flow: your code, or the model. Workflows are predictable and easy to debug. Agents are flexible but harder to reason about. Most production systems that people proudly call agents are actually workflows, and that is usually the right choice.

2. The real bottleneck is the harness, not the model

A popular shorthand is that an agent is just an LLM, plus tools, plus a loop. That definition is true but misleading, because it leaves out memory, planning, context, and guardrails, which are exactly the parts that separate agents that ship from agents that only demo.

A useful mental model: the model is the brain, and the harness is the body. The harness is everything you build around the model: how you assemble what it sees each turn, the tools you expose, how you store and retrieve memory, how you cap runaway behavior, and how you measure quality. In 2026 this work has its own name, agent engineering, and it is where most of your time will go.

The biggest shift in the last two years is from prompt engineering to **context engineering**. The wording of a single prompt matters far less than the full set of information the model sees on each turn: retrieved facts, prior results, tool outputs, and constraints. Get the context right and an average prompt works. Get it wrong and no amount of prompt wordsmithing saves you.

3. Start simpler than you think

There is a spectrum of agentic systems, from fully deterministic to fully autonomous. Reach for the simplest pattern that solves your problem, and only add autonomy when a simpler pattern genuinely cannot do the job. Every step toward autonomy buys flexibility and costs predictability.

Pattern	What it is	Reach for it when
Single prompt	One model call with good context. No tools, no loop.	The task is one well-defined step, like classification or drafting.
Prompt chain	Output of one call feeds the next, in a fixed order.	The task breaks into clear sequential stages you can name in advance.
Routing	A classifier sends input to one of several specialized paths.	Inputs fall into distinct categories needing different handling.
Parallelization	Run several calls at once, then aggregate or vote.	Subtasks are independent, or you want multiple takes on one problem.
Orchestrator-worker	A lead model breaks work into subtasks and delegates them.	Subtasks are not known up front and must be decided at runtime.
Autonomous agent	The model loops freely with tools until the goal is met.	The path is open-ended and genuinely cannot be pre-scripted.

A rule of thumb: if you can draw the flowchart in advance, build a workflow. If the flowchart depends on what the model discovers along the way, you need an agent.

4. The building blocks of an agent

Every real agent is assembled from five parts. Learning to think in these terms makes any framework or codebase easier to read.

- **Model.** The reasoning engine. Claude, GPT, Gemini, or an open model such as Llama or Qwen. Pick for capability and cost, but treat it as swappable.
- **Loop.** The control structure that lets the model act, observe the result, and act again. Without a loop you have a single call, not an agent.
- **Tools.** Functions the model can call to do things: search the web, query a database, run code, hit an API, send a message. Tools are how the agent affects the world.
- **Memory.** Short-term memory holds the current task context. Long-term memory persists facts and past results across sessions, often in a vector store or plain files.
- **Context.** What you actually place in the model's window each turn. This is assembled, not given, and managing it well is most of the job.

5. The agent loop, step by step

The most common and most reliable starting pattern is ReAct, short for Reasoning and Acting. It remains the default base loop in 2026 because it is simple, well supported, and teaches you the fundamentals. The loop runs like this:

1. **Receive the goal.** The user states what they want. You add it to the context along with your system instructions and the list of available tools.
2. **Reason.** The model thinks about the next step given everything it can see.
3. **Act.** The model either calls a tool with specific arguments or decides it is finished.
4. **Observe.** You run the tool, capture the result, and add that result back into the context.
5. **Repeat.** Go back to the reasoning step with the new information. Continue until the model produces a final answer or you hit a stopping condition.

That is the whole engine. Everything else is refinement: better tools, smarter context assembly, memory, and the safety rails in section 10. Writing this loop yourself once, even in fifty lines, teaches you more than any framework tutorial.

6. Build it yourself or use a framework?

This is the decision people agonize over most. The honest answer: write the loop yourself first so you understand it, then adopt a framework only when you feel real pain that the framework removes.

Rolling your own means calling the model SDK directly, defining tools as plain functions, and writing the loop. You get full transparency and no lock-in. The cost is that you build orchestration, retries, and state handling yourself.

Using a framework gives you prebuilt orchestration, state management, and multi-agent plumbing. The cost is a learning curve, some magic you cannot see, and occasional lock-in. The leading options in 2026:

Framework	Best for	Main tradeoff
LangGraph	Complex flows with branching and loops, modeled as an explicit graph you can inspect and debug.	More setup than simple agents need.
CrewAI	Role-based teams of agents that collaborate, with an approachable mental model.	Less fine-grained control over execution.
AutoGen	Asynchronous, conversational multi-agent coordination.	Can get complex fast for simple jobs.
OpenAI Agents SDK	Clean, opinionated single and multi-agent apps using the handoff pattern.	Locked to OpenAI models, limited portability.
Semantic Kernel	Enterprise apps on the Microsoft and .NET stack.	Heavier, oriented to large organizations.

Whatever you choose, adopt the Model Context Protocol (MCP) for connecting tools and data sources. MCP has become the common standard for exposing tools to agents, which means you can reuse integrations instead of hand-wiring each one to each model.

7. Tools are your highest-leverage investment

An agent is only as good as the tools you give it. A capable model with badly designed tools will struggle on simple tasks, while a modest model with well-designed tools can be remarkably effective. Spend real time here. A few rules that pay off:

- **Write tool descriptions for the model, not for you.** Say plainly what the tool does, when to use it, and what each argument means. The model reads these descriptions to decide what to call.
- **Return results the model can act on.** Shape outputs into clean, relevant text or structured data. Do not dump raw API responses with fifty irrelevant fields.
- **Make errors recoverable.** When a tool fails, return a clear message the model can read and adjust to, rather than a stack trace or a silent failure.
- **Keep the toolset small.** Five sharp tools beat twenty overlapping ones. Too many choices make the model pick wrong.
- **Prefer search then read.** For information gathering, let the agent search, pick the best result, then fetch it. Do not let it guess URLs or addresses.

8. Context engineering in practice

Context is the model's entire view of the world on a given turn, and it is finite. Managing it well is the craft that most separates strong agents from weak ones. In practice this means:

- **Curate, do not dump.** Put in what the current step needs and leave out the rest. More context is not better. Irrelevant material actively degrades reasoning and raises cost.
- **Retrieve selectively.** Pull only the facts relevant to the task from memory or your knowledge base, rather than stuffing everything in.
- **Compress history.** On long runs, summarize earlier steps instead of carrying every raw message forward. This keeps the window focused and controls cost.
- **Isolate context across sub-agents.** When you split work across multiple agents, give each its own clean context so they do not pollute one another. This is exactly how systems like Claude Code and Cursor run subagents.

9. When to use multiple agents

Multi-agent setups are fashionable and frequently premature. A single well-built agent with good tools beats a fragile committee of agents that pass control around and lose the thread. Reach for multiple agents only when the work genuinely benefits from separation.

The two patterns that hold up in production are **orchestrator-worker**, where a lead agent decomposes a task and delegates pieces to specialized workers, and the **handoff pattern**, where a triage agent routes to a specialist that can solve the request or pass it onward. Both work well up to a point. Past roughly eight to ten agent types, handoffs tend to become unwieldy and hard to debug. Add agents the way you would add microservices: reluctantly, and only when the boundaries are real.

10. Making agents reliable

This is the section everyone skips and later regrets. In 2026 the whole game is reliability. Proving an agent works could be done in 2025; making it work consistently is the current bar. Build these in from the start, not after launch.

- **Write evaluations early.** Build a test suite of real tasks with expected outcomes before you scale. Without evals you are tuning blind, and every change is a guess.
- **Cap the loop.** Set a maximum number of iterations. Agents get stuck in search-and-refine loops forever otherwise.
- **Budget tokens and cost.** Track spend per run and set hard limits. Runaway loops can quietly burn real money.
- **Add guardrails.** Validate inputs and outputs, restrict what tools can touch, and require human approval for sensitive or irreversible actions.
- **Instrument everything.** Log each step, tool call, and decision so that when a run goes wrong you can see why. Observability is not optional for systems that act on their own.
- **Fail gracefully.** Decide what the agent does when a tool errors or it cannot finish. Escalate to a human rather than guessing on high-stakes steps.

11. A practical path to your first agent

If you want a concrete order of operations, follow this:

1. Pick one narrow, repetitive task you understand well, such as triaging support tickets or compiling a research brief.
2. Ask whether a workflow would do. If you can script the steps, build that and stop here.
3. If the task needs runtime decisions, write a minimal ReAct loop yourself against the model SDK.
4. Add one tool at a time. Test after each. Resist the urge to add ten at once.
5. Build a small eval set of real cases as soon as the agent does anything useful.
6. Add memory only once you see the agent forgetting things it needs across turns.
7. Add loop caps, cost limits, and guardrails before anyone but you uses it.

8. Adopt a framework only when your hand-rolled version hits a wall the framework clearly solves.
9. Consider a second agent only when one agent with good tools provably cannot do the job.

12. Common pitfalls

- Building an autonomous agent when a workflow would be simpler, cheaper, and more reliable.
- Obsessing over prompt wording while ignoring what context the model actually receives.
- Giving the agent too many tools, or tools with vague descriptions and messy outputs.
- Shipping without evals, then having no way to tell if a change made things better or worse.
- Forgetting iteration and cost limits, then watching an agent loop forever.
- Reaching for multi-agent architectures before a single agent has been made to work.
- Letting the agent take irreversible actions, such as deletions or payments, without a human in the loop.

Where to go next

If you take one thing from this guide, make it this: build the smallest deterministic thing that works, invest in context and tool quality over model choice, and earn every increment of autonomy with an evaluation that proves it is safe to add.

Good starting points for deeper reading are Anthropic's writing on building effective agents, the documentation for whichever framework you choose, and the Model Context Protocol specification for tool integration. Then build something small and real. The fastest way to understand agents is to ship one.

This guide reflects the state of agent development as of May 2026. The tooling moves quickly; the principles move slowly.